

UAM OVERDRACHT • INTERN TECHNISCH DOSSIER

UAM legacy overdrachtdossier

Legacy agent, IST/DOEL-samenvattingen, C#-doelarchitectuur en migratieroadmap

Versie 1.0 • 23 juli 2026

Bronnen gefixeerd met SHA-256 • geen tokens of wachtwoorden opgenomen

UAM IST in een oogopslag

Doel van dit blad: in circa één A4 begrijpen wat de huidige UAM-oplossing doet, hoe zij technisch is opgebouwd en waar de belangrijkste overdrachtsrisico's zitten.

Wat draait er nu?

De endpointagent is een PowerShell-script van **4.805 regels** dat met PS2EXE als `uam.exe` zonder console wordt uitgevoerd. Voor debugging bestaat een consolevariant. Naast de executable is `dbsettings.txt` nodig; daarin staat de MSSQL-connectieroute versleuteld. Tijdens de start haalt de agent globale instellingen, eventuele gebruikersafwijkingen, filters en eerder opgeslagen checkpoints uit de database.

De agent voert hoofdzakelijk drie collectors uit: **browserhistorie**, **Windows Recent/Quick Access** en **Windows-processen**. Hij controleert per collector of deze actief is en opnieuw moet draaien, verzamelt gegevens, maakt detail- en minimale logging en actualiseert checkpoints. SQL-statements worden eerst in geheugen en CSV gebufferd en daarna transactiewijs rechtstreeks naar MSSQL verstuurd. Bij verbindingsproblemen wordt de CSV-buffer bij een volgende start hersteld.

Huidige componenten en gegevens

Onderdeel	IST
Endpoint	<code>uam.exe</code> , gecompileerde PowerShell; orchestratie, collectors, buffering en SQL in één proces.
Lokale configuratie	<code>dbsettings.txt</code> ; nog geen centrale lokale settingsdatabase.
Centrale database	27 tabellen, 569 velden, 13 primary keys, slechts 2 fysieke foreign keys.
Grootste historie	<code>DeviceBrowserLogging_Archive</code> : circa 2,47 miljoen rijen op het meetmoment.
Beheerportaal	PowerShell Universal-app voor gebruikersselectie, loggingdata, settings, uitsluitingen, applicatiematches, taken/planningen, errors en audit.
Bronnen	UAM-tabellen plus klantgebonden HR-/AD-structuren; veel relaties bestaan alleen impliciet in code en SQL.

Wat is waardevol en moet behouden blijven?

- Checkpoints per collector, deferred verwerking en herstel na tijdelijke storingen.
- Browser-tijdconversies, profiel-/bestandsstabiliteitscontroles en normalisatie van legacy settings.
- Een compacte minimale gebruiksregistratie naast detailgegevens.
- Doelgroep- en gebruikersafwijkingen, uitsluitingen/matches, foutcontext en auditinformatie.
- De bestaande functionele indeling van het PSU-portaal als vertrekpunt voor het nieuwe beheerportaal.

Belangrijkste knelpunten

- Eén groot proces combineert hosting, scheduling, collectors, SQL, foutafhandeling en self-restart.
- Endpoints schrijven direct naar MSSQL; de CSV bevat complete SQL-statements en is geen robuuste transactional outbox.

- Veel tekstkoppelingen, ontbrekende foreign keys en klantafhankelijke HR-/AD-tabellen maken wijziging en hergebruik lastig.
- Dubbele functies, dynamische SQL, handmatige garbage collection en een aangetroffen hardcoded `$Pwd` vragen om expliciete sanering; het geheim wordt niet meegeleverd.
- Het taken-/planningsdeel is functioneel nog niet af en arbitraire scripts vereisen strengere isolatie en autorisatie.

Bewijsstand: legacy agent, productie-DDL, rij-/veldprofielen, data dictionary en DEV-portaalschermen zijn vastgelegd. De laatste geautomatiseerde DEV-heropname en de PROD-browseropname vereisen nog een geldige handmatige PSU-login; bestaande DEV-beelden zijn privacygemaskeerd en uitsluitend via leesacties verkregen.

UAM gewenst doelbeeld in een oogopslag

Doel van dit blad: in circa één A4 begrijpen hoe de gewenste C#-oplossing eruitziet en welke ontwerpkeuzes de legacyproblemen oplossen.

Kern van het doelbeeld

De nieuwe UAM wordt geen één-op-één vertaling van PowerShell naar C#. Er komt een kleine, ondertekende **uam.exe bootstrapper** die controleert of de gewenste agentversie aanwezig is, pakketten veilig installeert, atomair wisselt en kan terugrollen. **uam-agent.exe** wordt een .NET Worker Service die alleen policy, planning, gezondheid en taakcoördinatie verzorgt.

Collectors worden afzonderlijke, versieerbare taskpackages: Browser History, Recent Items/Quick Access, Process Inventory en Browser Extensions. Eenvoudige volledig managed taken kunnen tijdelijk in een collectible **AssemblyLoadContext** draaien. Taken met native dependencies, PowerShell of een groter storings-/geheugenrisico draaien geïsoleerd in **uam-taskhost.exe**, zodat afsluiten aantoonbaar geheugen vrijmaakt.

Gewenste componenten en gegevensstroom

Onderdeel	Doelbeeld
Lokale opslag	SQLite in WAL-modus voor settingssnapshot, gebruikersafwijkingen, taskstate, checkpoints en transactional outbox.
Upload	Kleine idempotente HTTPS-batches, gecomprimeerd met gzip, met retries, jitter en backpressure.
Centrale verwerking	Ingestion API → duurzame queue → schaalbare workers; endpoints krijgen nooit directe databasetoegang.
Centrale database	PostgreSQL met partitionering als voorkeursbasis; TimescaleDB pas kiezen na een representatieve benchmark.
Beheer	Moderne adminportal via een control-plane API, typed policyrevisies, capability-RBAC en volledige audit.
Integraties	Configureerbare HR-/AD-views/adapters en een interne Application Registry met optionele FK naar de echte CMDB.

Functionele richting

- Maak **alles een taak** met een expliciet contract, versie, schedule, timeout, privacyclassificatie en uitvoerformaat.
- Vervang brede logging door configureerbare **URL- en procesallowlists**, rechtstreeks gekoppeld aan een applicatiesleutel uit de Application Registry/CMDB.
- Maak globale instellingen en afwijkingen typed, versioned en centraal publiceerbaar; de agent gebruikt een lokaal gevalideerd snapshot.
- Voeg structured logging, correlatie-ID's, OpenTelemetry en instelbare diagnostiek toe, tijdelijk en gericht per gebruiker/apparaat.
- Bouw privacy, minimale gegevensverwerking, retentie, redactie van secrets/PII en veilige standaardwaarden in het ontwerp in.

Schaal en betrouwbaarheid

Ontwerp en test voor minimaal **6.000 gebruikers** en gelijktijdige pieken. De SQLite-outbox voorkomt dat metingen verloren gaan; idempotency voorkomt dubbele verwerking; queue en workers vlakken pieken af. Meet

batchgrootte, compressie, wachtrijduur, database-inserts, storagegroei en herstel na langdurige uitval voordat een database-extensie of cloudproduct wordt gekozen.

Aanbevolen migratievolgorde

1. Leg gedrag, privacybeleid, eventcontracten en acceptatiematen vast.
2. Bouw bootstrapper, Worker Service, SQLite/outbox en package signing.
3. Lever Browser History als eerste verticale slice van endpoint tot portal.
4. Voeg overige collectors en geïsoleerde taskhost toe.
5. Bewijs schaal, retentie en herstel; bouw daarna het beheerportaal uit.
6. Draai een gecontroleerde parallelle pilot, migreer alleen bruikbare configuratie en bouw legacy daarna gefaseerd af.

Beoogd resultaat: een kleine en herstelbare endpointagent, een portable/open platform waar mogelijk, een aantoonbaar schaalbare ingestieketen en een sneller, veiliger en beter configureerbaar beheerportaal.

Legacy agent uam.ps1

Samenvatting

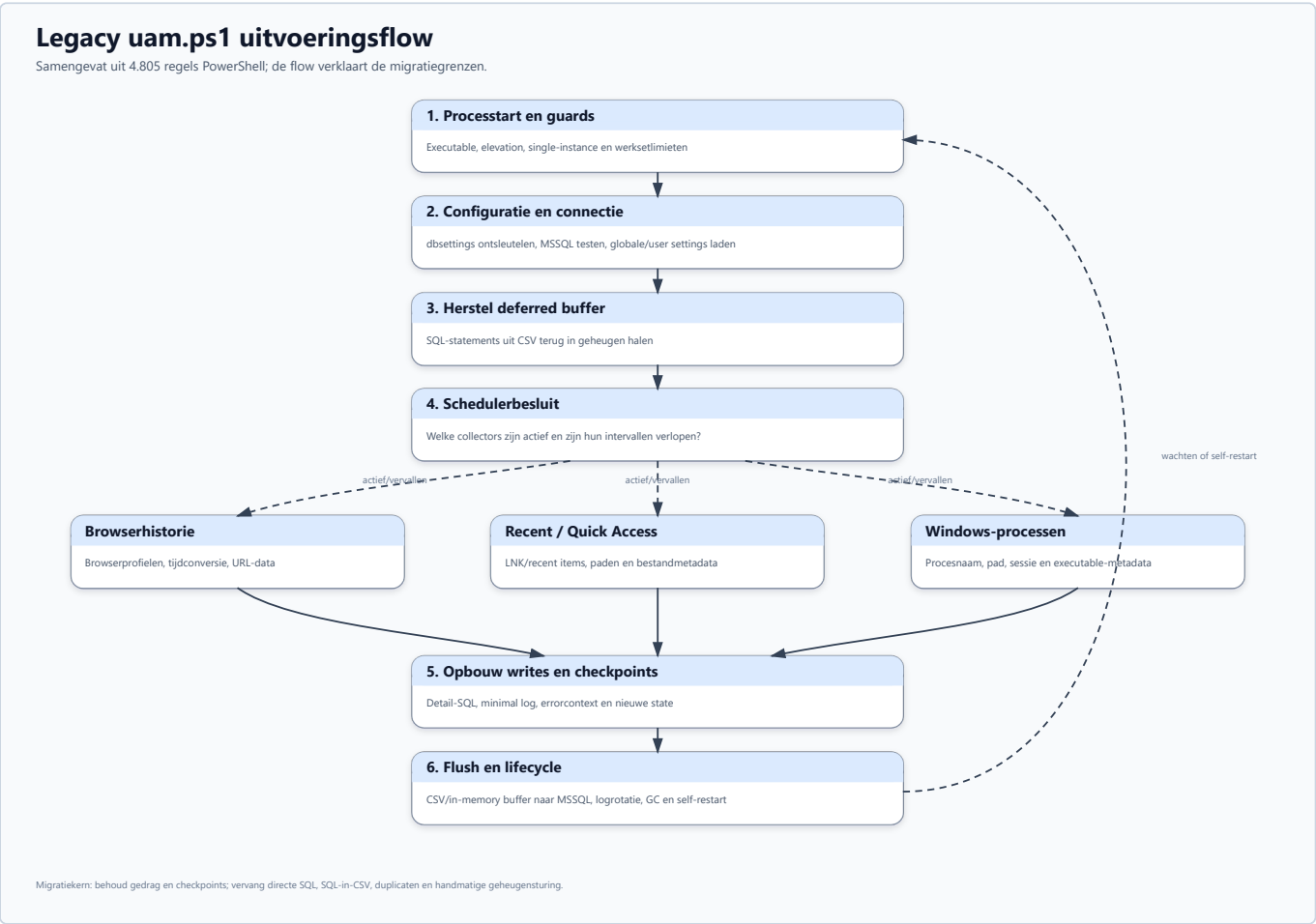
uam.ps1 is een monolithische, sequentiële PowerShell-agent. Eén proces verzorgt procesbewaking, configuratie, databaseverkeer, drie datacollectors, lokale buffering, foutafhandeling, logrotatie en self-restart. De functionele waarde zit vooral in de verzamellogica, checkpoints, uitsluitingen en foutcontext. De technische koppeling tussen al die verantwoordelijkheden moet niet één-op-één naar C# worden vertaald.

De productiebuild bestaat uit twee PS2EXE-varianten:

- `UAM.exe` : x64, zonder console, output/errorvenster onderdrukt;
- `UAM_console.exe` : x64 met console voor debugdoeleinden;
- bestandsversie: `0.7.0.0` ;
- runtimebestanden: executable plus `DbSettings.txt` ; voor SQLite kan daarnaast `winsqlite3.dll` of de fallback onder `sqlite\uam.exe` nodig zijn.

`DbSettings.txt` bevat de MSSQL-connectie-informatie via `ConvertTo/From-SecureString` met een vaste sleutel in de bron. Dat is obfuscatie, geen veilige secretopslag. De sleutel wordt bewust niet in dit document herhaald.

Uitvoeringsflow



De doorlopende hoofdlijn toont de lifecycle van het huidige proces. De drie zijtakken zijn de bestaande collectors. De gestippelde terugkoppeling maakt zichtbaar dat wachten, geheugencontrole en self-restart onderdeel zijn van de legacy lifecycle en niet van de toekomstige collectortaken horen te zijn.

Functiekaart

De regelnummers verwijzen naar de gefixeerde V0.7-bron.

Gebied	Belangrijkste functies	Regels	Betekenis voor migratie
Proces/start	Get-CurrentExecutableName, Test-IsElevated, Set-ProcesGeheugenBescherming	40-204	Bootstrap-/hostverantwoordelijkheid; niet in collectors opnemen.
Doelgroep/guards	Test-GroupMembership, Test-HasContent, Test-HasRows, Test-IsTrue, Test-IsDateTime	207-420	Normalisatie van slordige legacy instellingstypen is functioneel waardevol.
SQL-escaping	Format-SqlString, oude variant, Get-SafeSqlString	423-531, 1907-1941	Alleen als migratiebewijs gebruiken; nieuwe code moet parameters gebruiken.
Bestandslog	twee Manage-LogRotation, Write-Log	534-852	Duplicaat verwijderen; vervangen door structured logging en retention policy.
Telemetrie	Get-ProcessPerformance, Check-RAMandCPU	854-913, 1298-1323	Behoud signalen, niet de handmatige GC-/restartimplementatie.
Tijd/paden	browser-/epochconversies, Get-SafeStoragePath, waits	933-1295	Browser timestampconversies en profielbestandsstabiliteit blijven nodig.
MSSQL	Invoke-SqlQuery, Test-SqlConnection	1325-1627	Vervangen door parameterized API/outbox; endpoints schrijven niet direct naar SQL.
Buffer	Restore-DeferredQueriesFromCSV, Invoke-DeferredQueryProcessing	1629-1737	Functionele outbox behouden, complete SQL in CSV afschaffen.
Settings	Get-AppSetting	1739-1904	Typed settings, scope en defaults expliciet modelleren.
Fouten	Write-DetailedError, twee ErrorHandler, Write-Error2DB	1944-2339	Correlation/context behouden; secrets/PII redigeren en duplicaat verwijderen.
Lokale SQLite	Invoke-SQLiteQuery	2341-2397	Bewijst lokale SQLite-haalbaarheid; in C# Microsoft.Data.Sqlite gebruiken.
Procescoördinatie	Stop-OtherProcesses, Test-OtherProcessRunning	2439-2650	Vervangen door named mutex/service single-instance-contract.
Minimal model	New-MinimalLoggingQueryBlock	2663-2694	Functionele aggregatie behouden, maar server-side/idempotent uitvoeren.
Browser	Export-BrowserHistory2DB, Get-BrowserHistory, exclude update	2700-2989	Aparte BrowserHistory-taskplugin.
Recent files	Export-RecentFilesAndFolders2DB, Get-RecentFilesAndFolders	2996-3232	Aparte RecentItems-taskplugin.
Processen	Export-WindowsProcesses2DB, Get-WindowsProcesses, exclude update	3239-3504	Aparte ProcessInventory-taskplugin.
Orchestratie	Invoke-PreStart, Start-Logging	3511-4760	Opsplitsen in bootstrapper, agent host, scheduler en task executor.

Collectors

Browserhistorie

De agent ondersteunt Firefox, Edge en Chrome. Hij lokaliseert profielbestanden, wacht tot bestanden benaderbaar zijn, leest SQLite en vertaalt browser-eigen tijdstempels naar lokale tijd. Browsercheckpoints worden per browser in `DeviceLoggingUsers` bijgehouden. Er is ook een later geconsolideerd browsercheckpoint, waardoor dubbel modelleren is ontstaan.

Behoud in C#:

- profiel- en browserdetectie;
- veilig lezen van gelockte databases via een tijdelijke snapshot/copy;
- expliciete timestampconversie;
- per taak/per gebruiker/per apparaat een checkpoint;
- normalisatie van domein, URL en applicatiematch.

Wijzig in C#:

- log alleen URL-/domeinpatronen die op een centraal allowlist-item matchen;
- verwijder of hash querystring, fragment en gevoelige padsegmenten vóór opslag;
- koppel een match aan een interne `ApplicationId` en optioneel een externe CMDB-sleutel;
- gebruik parameterized SQLite en HTTPS-ingestie.

Windows Recent/Quick Access

De agent leest `.lnk`-bestanden en registreert meerdere bestandstijdstempels, doelpad en attributen. Dit levert nuttige gebruikssignalen op, maar paden kunnen persoons- of zaakgegevens bevatten.

De nieuwe taak moet een expliciet dataminimalisatiebeleid hebben: wel applicatie-/shareclassificatie, niet standaard de volledige bestandsnaam of het volledige pad.

Processen

De collector beperkt zich tot processen in de huidige Windows-sessie en registreert onder andere starttijd, executablepad, product en company. De bestaande uitsluitlijst is denylist-gebaseerd.

De nieuwe variant moet allowlist-gebaseerd zijn en bij voorkeur op meerdere kenmerken matchen: genormaliseerde procesnaam, ondertekende publisher, productnaam, optioneel padpatroon en interne applicatiesleutel.

Incidentele taken

Browserextensie-inventarisatie past functioneel als aparte taskplugin. Een generieke "voer willekeurige PowerShell uit"-plugin vergroot het aanvalsoppervlak sterk. Als zo'n compatibiliteitstaak tijdelijk nodig is, voer hem dan out-of-process uit met een ondertekend manifest, expliciete capability allowlist, timeout, outputlimiet en centrale kill switch.

Settings en checkpoints

`DeviceLoggingSettings` is een globale key/value-store met beschrijving, groep, waarde, eenheid, type en enabled-vlag. `DeviceLoggingUserSettings` legt afwijkingen vast voor gebruiker/domein/apparaat. `DeviceLoggingUsers` combineert identiteit, runtimegegevens, installatieversie, loggingstatus, geheugenmetingen en veel taakcheckpoints.

Deze semantiek moet in C# worden gescheiden:

- settings snapshot met schema- en revisienummer;
- user/device overrides met geldigheidsduur;
- task definition en task schedule;
- task checkpoint per `DeviceId + UserId + TaskId`;
- agent installation/runtime state;
- diagnostics policy.

De lokale SQLite-database is een cache en outbox, niet de enige bron van globale waarheid. Iedere centrale snapshot krijgt een revision/hash; de agent wisselt atomisch naar een volledig gevalideerde snapshot.

Legacy buffer en MSSQL-write

De agent bouwt complete `INSERT` -/ `UPDATE` -statements als strings op, bewaart die in memory en CSV en voert ze later in één transactie uit. Dit beperkt directe SQL-druk, maar mengt data en uitvoerbare code.

Belangrijkste problemen:

- waarden worden via stringconcatenatie in SQL geplaatst;
- databaseacties gebruiken op plekken `Invoke-Expression`;
- de CSV kan complete SQL plus gevoelige pad-/URL-/foutwaarden bevatten;
- een foutpad verwijdert het CSV-bestand en kan daarmee data verliezen;
- replay/idempotency is niet formeel vastgelegd;
- endpoints hebben directe MSSQL-connectiviteit en databasecredentials nodig.

De vervanger is een SQLite-outbox met records, niet SQL: immutable event-id, task-run-id, schema version, payload, created time, attempts en next-attempt. De server accepteert idempotente batches via HTTPS en verwerkt ze achter een queue.

Minimal logging

Naast detailtabellen houdt de agent `uam_log_minimal` en `uam_log_minimal_dates` bij. De intentie is waardevol: één logisch datapunt plus de dagen waarop dat punt is gebruikt. De implementatie laadt echter eerder minimal-historie voor een gebruiker in geheugen en de natuurlijke uniciteit wordt niet door een unieke constraint beschermd.

In het nieuwe model:

- bepaal een canonical key/hash uit tenant, user, application/data type en genormaliseerde waarde;
- gebruik server-side upsert met idempotency;
- maak de natuurlijke combinatie uniek;
- schrijf gebruiksdagen als unieke `(MinimalLogId, UsedOnDate)` -relatie of leid ze uit events af.

Fouten en observability

De agent bewaart bruikbare context: computer, gebruiker, loglevel, fout, script, regel, positie, callstack, commandline, versie, settingsnapshot en Windowsversie. Die context moet behouden blijven, maar velden als commandline, settings, URL, path en callstack moeten vóór transport worden geredigeerd.

Aanbevolen niveaus:

- `Information`: lifecycle, taakstatus, batchaantallen;
- `Debug`: timings en beslispaden zonder payload;
- `Trace`: tijdelijk, centraal geautoriseerd per gebruiker/apparaat en automatisch vervallend;

- `Warning/Error/Critical` : structured exception met correlation-, task-run- en device-id.

Gebruik OpenTelemetry voor metrics/traces en structured JSON logs voor supportbundles.

Concrete technische risico's

1. SQL-stringconcatenatie en dynamische uitvoering.
2. Vaste SecureString-sleutel in de bron.
3. Complete SQL-statements en gevoelige data in CSV.
4. CSV-verwijdering in een foutpad met mogelijk dataverlies.
5. Handmatige GC, vaste working-setlimieten van circa 50-200 MB en self-restart als geheugenbeheersing.
6. Mogelijke `RunAs /UAC`-prompt bij self-restart.
7. Dubbele definities van `Manage-LogRotation` en `ErrorHandler` ; de laatste definitie wint stilzwijgend.
8. Een browsercheckpointquery mist op één plek `userdomain` en kan de verkeerde samengestelde identiteit raken.
9. Geen database-uniekheid voor de natuurlijke minimal-logcombinatie.
10. Volledige eerdere minimal-historie per gebruiker wordt in geheugen geladen.
11. Hardcoded klanttabellen en HR/AD-aannames.
12. Eén proces bezit alle verantwoordelijkheden en native dependencies.

Wat zeker behouden moet blijven

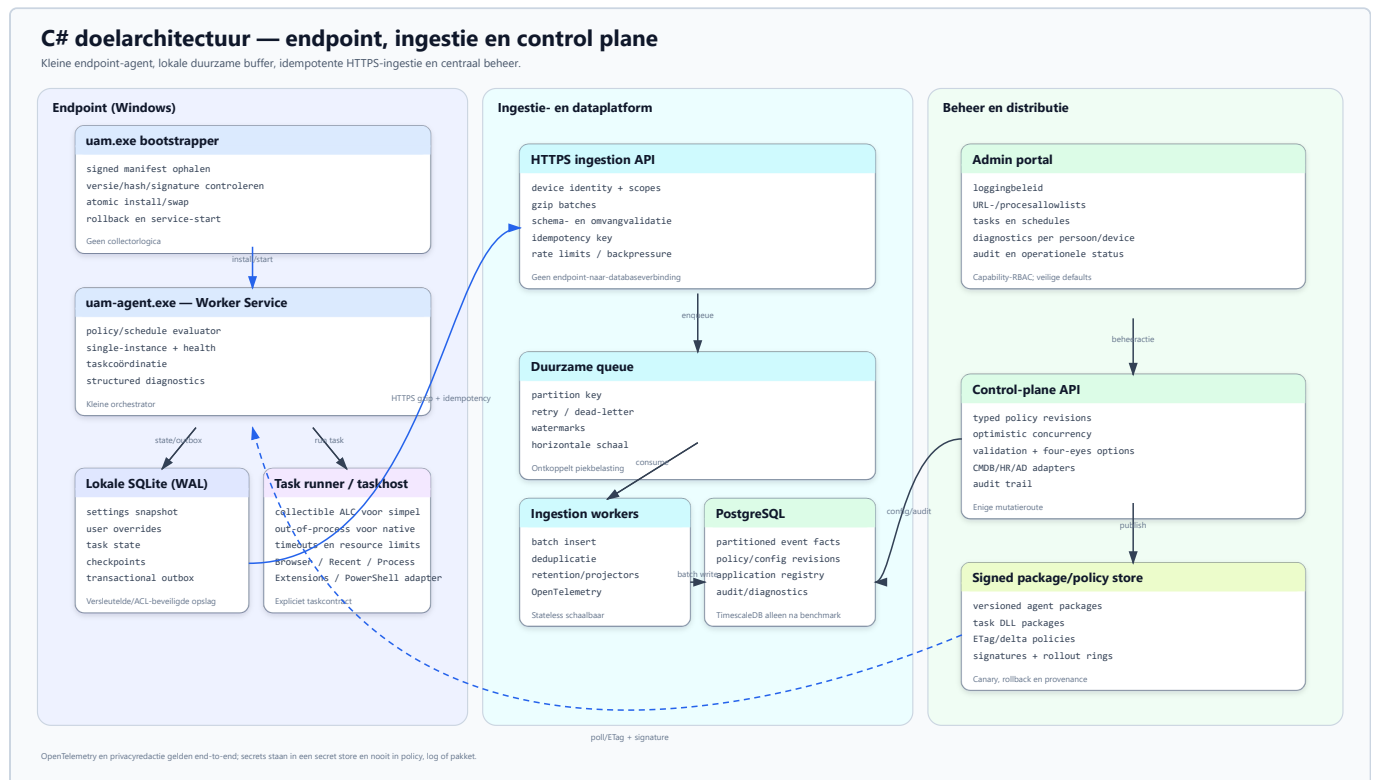
- expliciete taskcheckpoints en herstartbaarheid;
- local-first buffering bij netwerk- of serverstoringen;
- browser timestamp-/profielkennis;
- filtering vóór transport;
- per gebruiker/apparaat overrides;
- minimal/aggregated usage naast detaildata;
- rijke, centraal instelbare foutdiagnostiek;
- single-instance- en updatecontrole;
- duidelijke console- en stille productiemodus, vertaald naar service plus supporttool.

C# doelarchitectuur

Ontwerpdoelen

- kleine, stabiele agent met laag idle-geheugen;
- tasks afzonderlijk kunnen versiebeheerden, uitvoeren en isoleren;
- offline-first zonder dataverlies;
- 6.000 gebruikers zonder directe endpoint-naar-databasewrites;
- klantonafhankelijke HR/AD-/CMDB-contracten;
- allowlist en dataminimalisatie vóór transport;
- configureerbare diagnostics per gebruiker/apparaat;
- portable/open-source servercomponenten waar praktisch;
- veilige updates, rollback en aantoonbare audit.

Componentmodel



De drie zones scheiden endpointverantwoordelijkheden, data-ingestie en centraal beheer. De agent schrijft niet rechtstreeks naar een centrale database. Alle uploads lopen als idempotente batches via HTTPS; beleid en taakpakketten worden getekend, geversioneerend en gecontroleerd uitgerold.

1. Bootstrapper

Een kleine `uam.exe` bootstrapper doet alleen:

1. installroot en channel bepalen;
2. package manifest en digitale handtekening valideren;

3. ontbrekende/nieuwere `uam-agent.exe` downloaden of kopiëren;
4. uitpakken naar een versie-directory;
5. self-test uitvoeren;
6. atomisch de `current` -pointer wisselen;
7. service starten/health controleren;
8. bij failure naar vorige versie terugrollen.

Nooit een in-place overwrite van een draaiende executable. Houd minimaal `current`, `previous` en een begrensde packagecache. Het manifest bevat version, SHA-256, signing key id, minimum OS/runtime, files en rollout channel.

2. Agent host

Aanbevolen basis: .NET 10 LTS Worker Service als Windows Service.

Verantwoordelijkheden:

- device identity en registratie;
- policy/settings snapshots ophalen;
- scheduler en concurrencylimieten;
- taskpackage verifiëren;
- task runner starten;
- SQLite transacties/outbox;
- upload met retry/backpressure;
- health, metrics en supportbundle;
- updatehandoff naar bootstrapper.

Geen browserspecifieke SQL, HR-query's of portallogica in de host.

3. Taskcontract en isolatie

Een taskpackage bevat een signed manifest en één of meer assemblies:

```
public interface IUamTask
{
    Task<UamTaskResult> ExecuteAsync(
        UamTaskContext context,
        JsonElement settings,
        CancellationToken cancellationToken);
}
```

`UamTaskContext` geeft alleen capabilities die de taak nodig heeft: read-only lokale bestandstoegang via helpers, checkpointstore, eventwriter, clock en logger. Geen databascredential of willekeurige control-plane client.

Een collectible `AssemblyLoadContext` kan managed taskassemblies ontladen. Dit is geen harde isolatie: static references, threads, timers en native libraries kunnen unload blokkeren. Daarom:

- eenvoudige volledig managed tasks: collectible ALC in de agent;
- tasks met native SQLite/browserdrivers, onbekende code of PowerShellcompatibiliteit: apart `uam-taskhost.exe` -proces;
- per task timeout, memory/CPUlimiet, outputlimiet en cancellation;
- kill het taskhostproces wanneer echte vrijgave vereist is.

4. Lokale SQLite

Gebruik WAL-mode, korte transacties en één migratie-eigenaar. Encrypt gevoelige payloads waar het dreigingsmodel dat vraagt; bewaar geen serversecrets.

Voorgestelde tabellen:

Tabel	Sleutel/inhoud
agent_state	device id, install/current/previous version, registration en last contact
settings_snapshot	scope, revision, ETag/hash, schema version, JSON, activated/expiry
user_override	user key, setting/task key, value, source, valid from/until
task_definition	task id/version, manifest, schedule, enabled, capabilityset
task_checkpoint	device + user + task, cursor/watermark, last success/run
task_run	immutable run id, timings, result, counts, error fingerprint
outbox_batch	batch id, state, attempts, next attempt, size, idempotency key
outbox_event	event id, batch id, task/schema, occurred time, minimized payload
diagnostics_policy	level, scope, expiry, issuer en reason

De task schrijft events en checkpoint in één lokale transactie. Een checkpoint mag pas vooruit wanneer alle bijbehorende events duurzaam in de outbox staan.

5. Ingestion API

Gebruik ASP.NET Core achter TLS. Endpoints authenticeren als device/tenant met korte tokens of mTLS; geen MSSQL-/PostgreSQL-credentials op endpoints.

Voorbeeldcontract:

```
POST /v1/ingestion/batches
Authorization: Bearer <device-token>
Content-Encoding: gzip
Idempotency-Key: <batch-uuid>
X-UAM-Schema-Version: 1
```

Response:

```
{
  "batchId": "...",
  "status": "accepted",
  "acceptedEvents": 500,
  "retryAfterSeconds": 15
}
```

Serverregels:

- maximum compressed/uncompressed size en event count;
- schema-/tenant-/devicevalidatie vóór queue;
- idempotency op tenant + device + key;
- geen PII in accesslogs;

- 429 met jittered retry bij backpressure;
- poison batches isoleren met expliciete reden en support-id.

Startwaarden om te meten, niet als harde waarheid: maximaal 500 events of 1 MiB uncompressed per batch, maximaal twee uploads per device, exponential backoff met full jitter en een centraal instelbaar ratebudget.

6. Queue en workers

De API bevestigt pas na duurzame queue-opslag. Geschikte portable opties zijn RabbitMQ of NATS JetStream; een PostgreSQL-backed inbox kan voor een eerste kleine installatie volstaan. Houd het queuecontract abstract zodat deploymentprofielen kunnen verschillen.

Workers:

- valideren en normaliseren;
- matchen URL/proces naar `ApplicationId`;
- bulk-copy/upsert per eventtype;
- bewaken idempotency;
- vullen detailfeiten, minimal aggregates en diagnostics;
- schrijven dead-letter metadata zonder ongeredigeerde payload in algemene logs.

7. Databasekeuze

Aanbevolen default: PostgreSQL, omdat het open source, portable en geschikt voor partitionering/bulk ingestie is.

- partitioneer grote eventtabellen op tijd en tenant;
- gebruik BRIN voor lange tijdreeksen en gerichte B-tree-indexen voor tenant/device/application/time;
- scheid control-plane tabellen van high-volume facts;
- bewaar ruwe detaildata korter dan aggregaties;
- gebruik `COPY` /batch insert en voorkom row-by-row writes.

TimescaleDB kan nuttig zijn voor compressie, retention policies en time-bucket queries, maar voeg de extensie pas toe nadat echte loadtests aantonen dat standaard PostgreSQL-partitionering onvoldoende of operationeel onhandig is. MSSQL kan een deploymentadapter blijven, maar is geen vereiste voor de endpointarchitectuur.

8. Schaal naar 6.000 gebruikers

De architectuur voorkomt thundering herds:

- schedules krijgen stabiele device-jitter;
- policy kan per task globale concurrency/rate windows publiceren;
- lokale outbox absorbeert netwerk- en serveruitval;
- API geeft backpressure terug;
- queue ontkoppelt HTTP-latency van databasewrites;
- workers schalen horizontaal;
- databasewrites gebeuren in bulk.

Capaciteit moet worden berekend uit gemeten events per user/task/dag, gemiddelde payload, piekfactor, retention en query-SLA. Test minimaal:

1. steady state 6.000 actieve agents;
2. ochtendpiek en reconnect na één uur outage;

3. server/queue 30 minuten unavailable;
4. poison batch en schema mismatch;
5. agentupdate tijdens backlog;
6. databasepartition rollover en retention.

9. HR/AD-/CMDB-contracten

Definieer smalle contractviews in plaats van klanttabellen:

```
PersonSourceView
- ExternalPersonId
- AccountName
- Domain
- IsActive
- OrganizationUnitId
- ManagerExternalPersonId
- ValidFrom / ValidUntil

ApplicationSourceView
- ExternalApplicationId
- Name
- Owner
- LifecycleStatus
- SourceSystem
```

De connection/providerconfiguratie staat centraal en secrets zitten in een secret store. Een import registreert source system, schema version, batch id, row counts en kwaliteit. De interne application registry krijgt een eigen stabiele `ApplicationId` en optionele externe sleutels.

10. URL- en procesallowlists

Voorgesteld model:

- `application`;
- `application_external_key`;
- `url_match_rule` met host/padpattern, query policy en priority;
- `process_match_rule` met normalized name, publisher, product en padpolicy;
- `rule_revision` /audit;
- `match_conflict` voor ambiguïteit.

Match op het endpoint zodat niet-toegestane URL's/processen nooit in de outbox komen. Publiceer een compacte, gesigineerde rule snapshot. Test rules centraal met voorbeeldinput zonder echte gebruikersdata.

11. Diagnostics en privacy

- structured logs met event id, task run, device, appversion en correlation;
- OpenTelemetry metrics/traces;
- levels centraal en tijdelijk per user/device instelbaar;
- tracepolicy heeft issuer, reason en automatische expiry;
- redactie vóór lokale persistence en transport;
- supportbundle bevat configuratiehashes/counts, geen secrets/cookies/ruwe browserhistorie;
- retention en detailtoegang per tenant/purpose.

12. Admin portal

ASP.NET Core API plus een moderne webclient, met capability-RBAC:

- Viewer;
- Policy Administrator;
- Task Publisher;
- Task Operator;
- Diagnostics Operator;
- Auditor.

Portalmodules:

- fleet/agent health;
- population/policy editor met impact preview;
- task catalog, schedules en staged rollout;
- application registry en allowlists;
- usage explorer met aggregatie-first detail drilldown;
- errors/fingerprints en supportacties;
- immutable auditlog.

Iedere mutatie gebruikt optimistic concurrency, reason, actor en revision. Productieacties worden nooit verstoep in generieke tabel-row-actions.

Niet doen

- geen directe endpoint-naar-databaseconnectie;
- geen complete SQL-statements in SQLite/CSV;
- geen fixed-key secretversleuteling;
- geen willekeurige task-DLL zonder signature/capabilities;
- niet aannemen dat `AssemblyLoadContext` native resources hard isoleert;
- geen volledige URL/pad/processen verzamelen en later pas filteren;
- geen klanttabellen hardcoden;
- geen "Timescale omdat het time series heet" zonder benchmark.

Migratiekaart en roadmap

Legacy naar doelcomponent

Legacy	Nieuwe verantwoordelijkheid	Migratievorm
PS2EXE UAM.exe	uam.exe bootstrapper + Windows Service uam-agent.exe	Nieuwe implementatie; legacy alleen als gedragsspecificatie.
Invoke-PreStart	bootstrap/install/policy initialization	Opsplitsen in idempotente lifecyclestappen.
Start-Logging	scheduler + task orchestrator	Nieuwe state machine.
Browserfuncties	Uam.Task.BrowserHistory	Vertical slice als eerste collector.
Recent-itemfuncties	Uam.Task.RecentItems	Privacybeleid vóór eventwrite.
Procesfuncties	Uam.Task.ProcessInventory	Allowlist/publisher matching.
Browserextensiewens	Uam.Task.BrowserExtensions	Aparte incidentele/periodieke plugin.
DeviceLoggingSettings	centrale typed policy + lokale snapshot	Migreren met schema/revision.
DeviceLoggingUserSettings	scoped override policy	Normaliseren op user/device/task.
DeviceLoggingUsers	device/user identity, task checkpoint, runtime health	Opsplitsen in meerdere tabellen/entities.
Deferred SQL CSV	SQLite transactional outbox	Niet converteren naar SQL; alleen records/events.
Invoke-SqlQuery endpoint	HTTPS ingestion client	Databascredentials verwijderen.
Active/archive logtabellen	gepartitioneerde eventfacts + retention	Dual-write/backfill waar nodig.
Minimal tables	idempotente aggregate projector	Natuurlijke unique key toevoegen.
DeviceScripts	signed task package catalog	PowerShell alleen als begrensde compatibiliteit.
DeviceScriptSchedules	scheduler policy + target relations + run history	Gedenormaliseerde lijsten vervangen.
DeviceApplicationMatches	application registry + URL/process rule revisions	Externe CMDb-key behouden.
DeviceLoggingErrors	structured diagnostics/fingerprint store	Redactie en correlation.
ActionLog	immutable typed audit events	Polymorf TableName/RecordID afbouwen.
PSU-app	nieuwe admin portal/control-plane API	Capability-RBAC en revisions.

Fases

Fase 0 — baseline en governance

- bronhashes en database-DDL bevriezen;
- event-/setting-/taskcontracten vastleggen;
- gegevensclassificatie, doeleinden en retention goedkeuren;
- 20-50 representatieve pilotdevices kiezen;
- legacy metrics verzamelen: eventvolume, backlog, memory, foutpercentages.

Exit: goedgekeurde contracts v1 en meetbare legacybaseline.

Fase 1 — platform skeleton

- repository/CI, signing en package manifest;
- bootstrapper met atomic install/rollback;
- agent Windows Service;
- SQLite migrations, task state en outbox;
- control-plane heartbeat/settings endpoint;
- structured logs en supportbundle.

Exit: agent kan veilig installeren/updaten, policy ophalen en offline events vasthouden.

Fase 2 — verticale BrowserHistory-slice

- Chromium/Firefox reader;
- URL-allowlist en redactie op endpoint;
- task checkpoint + outbox in één transactie;
- HTTPS batch ingestie, queue en PostgreSQL worker;
- minimale portalview voor fleet/ingestion.

Exit: end-to-end pilot zonder directe SQL-connectie en zonder dataverlies bij outage.

Fase 3 — overige collectors

- ProcessInventory met publisher/product matching;
- RecentItems met padminimalisatie;
- BrowserExtensions als aparte task;
- taskhostproces voor native/risicovolle tasks;
- centraal taskcatalogus-/schedulecontract.

Exit: functionele pariteit voor de drie actieve legacy collectors plus extensie-inventarisatie.

Fase 4 — schaal en datamodel

- 6.000-agent load/reconnecttests;
- databasepartitionering, indexes en retention;
- queue/worker autoscaling en dead-letter flow;
- idempotent minimal aggregate;
- HR/AD-/CMDB-viewadapters.

Exit: capaciteitstest voldoet aan overeengekomen ingestie-, backlog- en query-SLA.

Fase 5 — beheerportal

- population/policy editor;
- agent health en diagnostics;
- task catalog/schedules/targeting;
- application registry en allowlists;
- usage explorer, errors en audit;
- impact preview, approval en rollback.

Exit: alle normale beheerhandelingen kunnen zonder PSU en met capability-RBAC.

Fase 6 — parallel pilot en uitrol

- legacy en nieuw naast elkaar op pilots;
- compare aggregates, niet klakkeloos gevoelige detailregels;
- canarygroepen 1%, 5%, 20%, 50%, 100%;
- automatische rollbackgrenzen;
- support/runbooks en eigenaarsoverdracht.

Exit: datakwaliteit, performance, privacy en operationele acceptatie zijn ondertekend.

Fase 7 — legacy afbouw

- endpoint-MSSQL-credentials intrekken;
- legacy schedules/PSU-writefuncties uitschakelen;
- dataretentie/backfillbesluit uitvoeren;
- oude executables, CSV-buffer en fixed-key DbSettings verwijderen;
- legacy portal read-only maken en daarna uitschakelen.

Datamigratie

Niet alle detailhistorie hoeft naar het nieuwe operationele model. Beslis per dataset:

Dataset	Voorkeur
Global settings en geldige overrides	Transformeren naar typed policy v1, met bron/provenance.
Application matches	Opschonen en migreren naar application/rule revisions.
Process/browser excludes	Alleen als input voor allowlistontwerp; niet blind kopiëren.
Task/scripts/schedules	Handmatig reviewen; geen arbitraire scriptcode automatisch activeren.
Minimal logs/dates	Backfill naar aggregate model indien rapportagehistorie nodig is.
Detail archives	Bij voorkeur read-only legacy/archive store; selectieve backfill na businesscase.
Errors/audit	Bewaren volgens compliance; eventueel in apart legacy auditarchief.
HR/AD tabellen	Niet migreren als UAM-data; vervangen door broncontracten/views.

Veldreview

De dictionary markeert vier kandidaten, geen verwijderbesluiten:

- `DeviceBrowserLogging2Exclude.Remarks` ;
- `DeviceScriptSchedules.IntervalMinutes` ;
- `DeviceScriptSchedules.SpecificRunTime` ;
- `DeviceScriptSchedules.LastRunTime` .

Beslis na controle op externe rapporten, ad-hoc SQL en gewenst nieuw schedulercontract. De vele externe `AdUsers` -/HR-velden worden niet individueel “verwijderd”; de nieuwe bronview projecteert uitsluitend benodigde velden.

Acceptatiecriteria

Agent

- idle- en piekgeheugen binnen afgesproken budget;
- crash/reboot verliest geen geaccepteerde lokale events;
- taskcheckpoint loopt nooit vooruit op outboxcommit;
- update kan automatisch rollbacken;
- tasks kunnen worden gestopt en native taskhostresources verdwijnen.

Ingestie

- duplicate batch levert geen duplicate facts;
- outage/reconnect voldoet aan backlog recovery SLA;
- 429 /retry veroorzaakt geen thundering herd;
- ongeldige schema-/tenant-/devicepayload wordt deterministisch afgewezen;
- geen raw payload/secrets in platformlogs.

Data/privacy

- niet-allowlisted URLs/processen verlaten het endpoint niet;
- URL query/fragment en paden volgen redactiebeleid;
- retention/partition purge is aantoonbaar;
- detailtoegang is scoped en geaudit;
- aggregates vergelijken binnen overeengekomen tolerantie met legacy.

Portal

- read en write capabilities zijn gescheiden;
- iedere mutatie heeft actor/reason/revision;
- impact preview en rollback zijn beschikbaar;
- diagnosticsbeleid verloopt automatisch;
- audit is immutable en doorzoekbaar.

Open ontwerpbesluiten

1. PostgreSQL-only basis of optionele TimescaleDB deployment.
2. Queuekeuze per deploymentprofiel.
3. Deviceauthenticatie: mTLS, managed certificate of korte tokenflow.
4. Exacte URL-/padminimalisatie per businessdoel.
5. Retentie detail versus aggregates.
6. In-process ALC versus out-of-process per tasktype.
7. Portaltechnologie en hostingmodel.
8. Welke legacy historie aantoonbaar moet worden gebackfilled.
9. Wie eigenaar is van de centrale application registry voor andere IAM-modules.

Eerste backlog

1. ADR's voor agent/service, SQLite/outbox, ingestion en database.

2. Contractpackages `Uam.Contracts` , `Uam.TaskSdk` , `Uam.Ingestion.Contracts` .
3. Bootstrapper prototype met signed package en rollback.
4. SQLite schema/migration en power-loss tests.
5. BrowserHistory vertical slice met allowlist.
6. Ingestion API + idempotency + queue worker.
7. 6.000-agent synthetic load harness.
8. Portal read-only fleet/usage prototype.
9. Policy revision/approval/audit flow.
10. Pilotrunbook en legacy comparison dashboard.